TRAINING EXERCISES

Wire Framework Developer Workbook.

Five hands-on exercises designed to be worked through during the half-day Wire onboarding workshop. Participants install the plugin from scratch, scaffold a real engagement, run a full generate → validate → review lifecycle, and practise picking release types against realistic SOW scenarios.

FORMAT	TOTAL TIME	SETUP	SOLUTIONS
Workbook	**~70 min**	**Claude Code**	**Included**
Self-paced + facilitated	Across 5 exercises	Open in any terminal dir	Page 6, Ex 4 & Ex 5

5	4	2
EXERCISES	SOW SCENARIOS	SOLUTION SETS
Install · scaffold · lifecycle · selection · diagnose.	Cover dashboard-only, discovery, pipeline-only and seed-first.	Answer keys for Exercises 4 and 5.

DOCUMENT CONTENTS

BEFORE YOU START

- All exercises assume Claude Code is open in a terminal.
- You'll need write access to a clean local directory for Exercise 2.
- Exercises build on each other — complete in order.
- Solutions are at the end. Try first before peeking.

STUCK?

Ask in `#wire-framework`, or flag the facilitator. Most issues are covered in the troubleshooting appendix of the session datasheet.

§1 Hands-on · setup

Ex 1
PLUGIN INSTALL

Install the Wire plugin.

10 min

Objective · get Wire running in Claude Code from scratch.

STEPS

1. Open Claude Code in any terminal directory:
```
claude
```
2. Register the Wire plugin marketplace:
```
/plugin marketplace add rittmananalytics/wire-plugin
```
3. Install the Wire plugin:
```
/plugin install wire@rittman-analytics
```
4. Activate the plugin in the current session:
```
/reload-plugins
```
5. Verify the install worked:
```
/wire:help
```

EXPECTED RESULT

You should see a full list of `/wire:*` commands organised by lifecycle phase.

QUESTIONS TO CONSIDER

- Difference between the Claude Code plugin and the Gemini CLI extension? Where do the command specs live in the source repo?

Ex 2
FIRST ENGAGEMENT

Create your first engagement.

15 min

Objective · initialise a Wire engagement and understand the directory structure it creates. **Scenario** · you've just been assigned a new client. You have a SOW PDF. Set up the Wire project.

SETUP

```
$ mkdir ~/wire-training-demo && cd ~/wire-training-demo && git init && claude
```

STEPS

1. Run `/wire:new`.
2. Answer the prompts — client `Training Co`, engagement `training_demo`, repo mode `A` (combined), release type `full_platform`, release ID `01-training-demo`, SOW path skip, issue tracker `none`.
3. Explore the structure: `find .wire -type f | sort`.
4. Read the generated status file: `cat .wire/releases/01-training-demo/status.md`.

EXPECTED RESULT

You should see `.wire/engagement/` and `.wire/releases/01-training-demo/` directories. The `status.md` contains YAML frontmatter listing all 15 `full_platform` artifacts, each set to `not_started`.

QUESTIONS TO CONSIDER

- What is the difference between `.wire/engagement/` and `.wire/releases/`? Which artifacts are marked `not_applicable`, and why?

§2 Hands-on · artifact lifecycle

Ex 3
FULL LIFECYCLE

Run a full requirements lifecycle.

25 min

Objective · experience the complete generate → validate → review gate cycle for a real artifact. **Scenario** · the SOW for Training Co describes a sales analytics platform. You need to produce a requirements specification.

SETUP — ADD MINIMAL ENGAGEMENT CONTEXT

```
$ cat > .wire/engagement/context.md << 'EOF'
# Engagement Context

**Client**: Training Co
**Engagement**: Sales Analytics Platform
**Objectives**: Build a sales performance dashboard in Looker on top of
  Salesforce data via Fivetran and BigQuery, with dbt models covering
  opportunities, pipeline, and win/loss analysis.
**Key Stakeholders**: VP Sales (primary), Sales Ops team (daily users)
**Data Sources**: Salesforce (Fivetran), HubSpot (Fivetran), Google Sheets quota
**Target Platform**: BigQuery + dbt Cloud + Looker
**Timeline**: 6 weeks
EOF
```

STEPS

1. Run requirements generate — watch the AI read the engagement context and produce a spec.
   ```
   /wire:requirements-generate 01-training-demo
   ```
2. Read what was generated:
   ```
   $ ls .wire/releases/01-training-demo/requirements/
   $ cat .wire/releases/01-training-demo/requirements/requirements_specification.md
   ```
3. Run validation, noting which checks ran and the pass/fail result:
   ```
   /wire:requirements-validate 01-training-demo
   ```
4. Run review. When prompted for reviewer feedback, type *"Approved — requirements look complete."*
   ```
   /wire:requirements-review 01-training-demo
   ```
5. Check what changed: `cat .wire/releases/01-training-demo/status.md | head -30` and the execution log `cat .wire/releases/01-training-demo/execution_log.md`.

EXPECTED RESULT

`requirements.generate`, `requirements.validate` and `requirements.review` should all show `complete` / `pass` / `approved` in `status.md`. The execution log should have three timestamped rows.

QUESTIONS TO CONSIDER

- What happens if validation fails? Try editing the requirements file to remove a section and re-run validate.
- What does the engagement-context skill output at the start of each command?
- Where does the requirements specification get saved?

§3 Group exercise · release type selection

Ex 4
SELECTION

Map SOW descriptions to release types.

10 min

Objective · map real-world SOW descriptions to the correct Wire release type. For each scenario below, identify the release type and list two or three artifacts that would be in scope.

SCENARIO A

Dashboards on existing dbt.

A client already has Fivetran, BigQuery, and a dbt project with 40 models. They want four new Looker dashboards built on top — no new data sources or model changes needed.

RELEASE TYPE _____
IN-SCOPE ARTIFACTS _____
OUT-OF-SCOPE _____

SCENARIO B

Uncertain scope.

A new client with uncertain scope. Stakeholders disagree on what the platform should do. You need to run a 1-week scoping sprint before any delivery work begins.

RELEASE TYPE _____
KEY ARTIFACTS _____
END COMMAND _____

SCENARIO C

New pipeline, existing BI.

A client needs a new Fivetran pipeline from Shopify → BigQuery, with dbt staging and integration models. No BI layer needed — they already have Looker.

RELEASE TYPE _____
IN-SCOPE ARTIFACTS _____
NOTES _____

SCENARIO D

Dashboards before data.

A client wants to see working dashboards in week 1 — before you've connected to their live database. They're happy to use seed data initially and refactor to real sources later.

RELEASE TYPE _____
KEY ENABLING ARTIFACT _____
TRANSITION COMMAND _____

TIP

Work in pairs. Each pair tackles two scenarios then trades partners — every scenario gets discussed by at least two people before the facilitator calls time. Solutions are on page 6.

§4 Diagnose · reading the execution log

Ex 5
LOG READING

Diagnose engagement state from the log.

10 min

Objective · diagnose the state of an engagement from its execution log alone.
Scenario · a colleague has left this log and you're picking up the engagement.

TIMESTAMP	COMMAND	RESULT	DETAIL
2026-04-01 09:15	skill · engagement-context	**activated**	Context loaded — release 01-acme-platform, full_platform.
2026-04-01 09:20	/wire:requirements-generate	**complete**	Generated requirements specification (13 sections).
2026-04-01 09:45	/wire:requirements-validate	pass	13 checks passed, 0 failed.
2026-04-01 10:00	/wire:requirements-review	**approved**	Reviewed by Sarah Chen.
2026-04-02 14:00	skill · engagement-context	**activated**	Context loaded — release 01-acme-platform, requirements approved.
2026-04-02 14:05	/wire:conceptual_model-generate	**complete**	Generated entity model with 7 entities.
2026-04-02 14:30	/wire:conceptual_model-validate	fail	2 issues: missing relationship between Order and Product; orphaned Discount entity.
2026-04-02 14:45	/wire:conceptual_model-generate	**complete**	Regenerated entity model — fixed 2 issues, 7 entities.
2026-04-02 15:00	/wire:conceptual_model-validate	pass	14 checks passed, 0 failed.

Q1 **Current state.**
What's complete, what's in progress, what hasn't started?

Q2 **Sessions.**
How many sessions have happened so far? How do you know?

Q3 **First-pass failure.**
Conceptual model failed validation on first pass — what was wrong, and was it fixed?

Q4 **Next command.**
What is the recommended next command to run?

Q5 **Lifecycle order.**
What artifact comes after `conceptual_model` in the `full_platform` lifecycle?

§S Solutions · Ex 4 & Ex 5

EXERCISE 4 · RELEASE TYPE SELECTION

A Dashboard extension

TYPE `dashboard_extension`
IN SCOPE requirements · mockups · dashboards · uat
OUT all pipeline and data-model artifacts — data already modelled.

B Discovery

TYPE `discovery`
ARTIFACTS problem_definition → pitch → release_brief → sprint_plan
END `/wire:release:spawn <discovery-release>`

C Pipeline only

TYPE `pipeline_only`
IN SCOPE requirements · pipeline_design · data_model · pipeline · dbt · data_quality · deployment

D Dashboard first

TYPE `dashboard_first`
ENABLING ARTIFACT `seed_data` — internally-consistent CSV fixtures so dbt and dashboards work before client data access.
TRANSITION `/wire:data_refactor-generate <release>`

EXERCISE 5 · LOG DIAGNOSIS

Q1 Current state

Requirements fully complete (approved). Conceptual model: generate and validate both complete (second pass passed); review not yet run. All other artifacts not started.

Q2 Sessions

Two sessions — 2026-04-01 (requirements) and 2026-04-02 (conceptual model). The `engagement-context` skill activation rows mark the start of each session.

Q3 First-pass failure

First validate found a missing relationship between Order and Product, and an orphaned Discount entity. Both fixed in the regeneration — second validate passed all 14 checks.

Q4 Next command

`/wire:conceptual_model-review 01-acme-platform`

Q5 Lifecycle order

`pipeline_design` — `full_platform` runs requirements → conceptual_model → pipeline_design → data_model → ...